CONCEPTOS DEL DÍA · SESIÓN 21

Funciones: invocación, retorno, alcance y bibliotecas

Documento de consulta para la persona estudiante

Módulo	CSTI12010 Diseño de algoritmos
Unidad de aprendizaje	UA2 · Empleo de lenguajes de programación en el paradigma de programación estructurada
Capacidad	2.3 Emplear funciones en la resolución de problemas según las especificaciones técnicas.
Saberes	Funciones: invocación y retorno · Variables globales y locales · Variables automáticas y estáticas · Funciones numéricas y alfanuméricas (math.h, ctype.h, stdlib.h)
Persona facilitadora	Giovanni Antonio Coto Calderón

Para qué le sirve este documento

Resume y amplía lo visto en la presentación. Úselo para repasar, hacer la práctica autónoma y prepararse para la Actividad de comprobación 2. Al final encontrará un glosario y preguntas de repaso.

1. ¿Qué es una función?

Un bloque con nombre que recibe datos (parámetros), hace una tarea y puede devolver un resultado (return). Se escribe una vez y se llama muchas veces.

iva.c

```c
#include <stdio.h>
#define TASA_IVA 0.13

double calcularIVA(double monto);       // prototipo

int main(void) {
    double precio = 15000;
    double iva = calcularIVA(precio);   // llamada
    printf("IVA de %.2f: %.2f\n", precio, iva);
    printf("IVA de 2500: %.2f\n", calcularIVA(2500));
    return 0;
}

double calcularIVA(double monto) {      // definicion
    return monto * TASA_IVA;
}
```

Salida real (gcc)

```
IVA de 15000.00: 1950.00
IVA de 2500: 325.00
```

Parte	Ejemplo
Prototipo	`double calcularIVA(double monto);`
Llamada	`iva = calcularIVA(precio);`
Definición	`double calcularIVA(double monto) { return monto * 0.13; }`
Sin resultado	`void mostrarMenu(void)`

2. Funciones void y reutilización

titulo.c

```c
#include <stdio.h>

void mostrarLinea(int largo) {
    int i;
    for (i = 1; i <= largo; i++) {
        printf("-");
    }
    printf("\n");
}

void mostrarTitulo(void) {
    mostrarLinea(19);
    printf("COMPROBANTE DE PAGO\n");
    mostrarLinea(19);
}

int main(void) {
    mostrarTitulo();
    return 0;
}
```

Salida real (gcc)

```
-------------------
COMPROBANTE DE PAGO
-------------------
```

leer.c

```c
#include <stdio.h>

int leerEntero(int minimo, int maximo) {
    int valor;
    do {
        printf("Valor (%d a %d): ", minimo, maximo);
        scanf("%d", &valor);
    } while (valor < minimo || valor > maximo);
    return valor;
}

int main(void) {
    int edad = leerEntero(15, 99);
    int opcion = leerEntero(0, 3);
    printf("Edad: %d  Opcion: %d\n", edad, opcion);
    return 0;
}
```

Salida real (gcc) · datos de entrada: 120, 20, 5, 2

```
Valor (15 a 99): 120
Valor (15 a 99): 20
Valor (0 a 3): 5
Valor (0 a 3): 2
Edad: 20  Opcion: 2
```

3. Alcance y duración de las variables

alcance.c

```c
#include <stdio.h>

int contador = 0;               // GLOBAL: la ven todas

void registrar(void) {
    int temporal = 10;          // LOCAL de registrar
    contador++;
    printf("registrar: contador=%d temporal=%d\n",
           contador, temporal);
}

int main(void) {
    int temporal = 99;          // LOCAL de main (otra)
    registrar();
    registrar();
    printf("main: contador=%d temporal=%d\n",
           contador, temporal);
    return 0;
}
```

Salida real (gcc)

```
registrar: contador=1 temporal=10
registrar: contador=2 temporal=10
main: contador=2 temporal=99
```

static.c

```c
#include <stdio.h>

void automatica(void) {
    int veces = 0;              // se crea en cada llamada
    veces++;
    printf("automatica: %d\n", veces);
}

void estatica(void) {
    static int veces = 0;       // conserva su valor
    veces++;
    printf("estatica:   %d\n", veces);
}

int main(void) {
    int i;
    for (i = 1; i <= 3; i++) {
        automatica();
        estatica();
    }
    return 0;
}
```

Salida real (gcc)

```
automatica: 1
estatica:   1
automatica: 1
estatica:   2
automatica: 1
estatica:   3
```

4. Bibliotecas de C

Biblioteca	Funciones	Ejemplo
math.h	sqrt pow fabs round ceil floor	pow(2, 10) → 1024
ctype.h	isalpha isdigit isspace toupper tolower	toupper('s') → 'S'
stdlib.h	rand srand abs atoi exit	rand() % 6 + 1

matematicas.c

```c
#include <stdio.h>
#include <math.h>

int main(void) {
    printf("sqrt(144)  = %.2f\n", sqrt(144));
    printf("pow(2, 10) = %.0f\n", pow(2, 10));
    printf("fabs(-7.5) = %.1f\n", fabs(-7.5));
    printf("round(2.5) = %.0f\n", round(2.5));
    printf("ceil(4.1)  = %.0f\n", ceil(4.1));
    printf("floor(4.9) = %.0f\n", floor(4.9));
    return 0;
}
```

Salida real (gcc)

```
sqrt(144)  = 12.00
pow(2, 10) = 1024
fabs(-7.5) = 7.5
round(2.5) = 3
ceil(4.1)  = 5
floor(4.9) = 4
```

caracteres.c

```c
#include <stdio.h>
#include <ctype.h>

int main(void) {
    char texto[] = "Ced 1-0234";
    int i, letras = 0, digitos = 0;
    for (i = 0; texto[i] != '\0'; i++) {
        if (isalpha(texto[i])) { letras++; }
        if (isdigit(texto[i])) { digitos++; }
        putchar(toupper(texto[i]));
    }
    printf("\nLetras: %d  Digitos: %d\n", letras, digitos);
    return 0;
}
```

Salida real (gcc)

```
CED 1-0234
Letras: 3  Digitos: 5
```

5. Errores frecuentes

Error	Qué pasa	Corrección
`Llamar sin prototipo`	implicit declaration (error en gcc 14+)	`Prototipo antes de main`
`Camino sin return`	Valor impredecible	`return en cada camino`
`Variable local en otra función`	'x' undeclared	`Parámetro o return`
`pow sin math.h`	implicit declaration	`#include <math.h>`

Glosario

Término	Definición breve
Argumento	Valor que se envía a una función en la llamada.
Alcance	Parte del programa donde una variable existe.
Función	Bloque con nombre que recibe datos, hace una tarea y puede devolver un valor.
Parámetro	Variable de la definición que recibe una copia del argumento.

Término	Definición breve
Prototipo	Declaración de una función antes de usarla.
return	Devuelve un valor y termina la función.
static	Variable local que conserva su valor entre llamadas.
void	Indica que no se devuelve (o no se recibe) nada.

Preguntas de repaso

1. ¿Para qué sirve el prototipo?
2. ¿Qué devuelve una función void?
3. ¿Dónde existe una variable local?
4. ¿Qué hace static en una variable local?
5. ¿Con qué función se calcula una potencia?
6. ¿Qué hace toupper('a')?

Fuentes recomendadas

- Kernighan, B. y Ritchie, D. (1991). El lenguaje de programación C (2.ª ed.). Prentice Hall.
- cppreference.com: funciones, alcance, almacenamiento static; math.h, ctype.h y stdlib.h.
- Programas de las sesiones 10, 17, 18 y 19 del curso.